

3. The "Rendezvous Point" – 25 pts (Wet)

Scenario: Two emergency response teams are located at different nodes in a city graph: Team Alpha at node n_1 and Team Bravo at node n_2 . They need to meet in a point R , a Rendezvous Point to consolidate equipment before a mission.

Objective: Find the node R such that the time it takes for both teams is minimized.

1. (7 pts) Provide the pseudocode of an algorithm of a function `findRendezvousPoint(n1, n2)` that solves the problem (i.e., return the node R).

Solution

```
findRendezvousPoint(n1, n2)
  assert n1 and n2 not null
  Map distTo1 = new_map()
  Map distTo2 = new_map()
  call dijkstraShortestPath(Graph G, n1)
  call dijkstraShortestPath(Graph G, n2)
  //dijkstraShortestPath will fill the distTo map with shortest path
  int bestDist = infity
  int candidate = null
  for each vertex in graph do
    if(max(distTo1[vertex],distTo2[vertex]) < bestDist)
      then bestDist = max(distTo1[vertex],distTo2[vertex])
        candidate = vertex
    end if
  end for
  return candidate
```

2. (8 pts) Show the algorithm is correct and provide its complexity. If your algorithm relies partially of algorithms given in the lecture, show can assume that those algorithms works correctly.

Solution

Here is the pseudocode, we use v to denote vertex for simplicity. We analyze its complexity first.

From lecture we know runtime complexity of `dijkstraShortestPath` is $O(|V|\log|V| + |E|\log|V|)$ where $|V|$ is the number of vertices and $|E|$ is the number of edges

Then

```

findRendezvousPoint(n1, n2)
  assert n1 and n2 not null
  //O(1) since we can build hash map
  Map distTo1 = new_map()
  Map distTo2 = new_map()
  //  $O(|V|\log|V| + |E|\log|V|)$ 
  call dijkstraShortestPath(Graph G, n1)
  //  $O(|V|\log|V| + |E|\log|V|)$ 
  call dijkstraShortestPath(Graph G, n2)
  //dijkstraShortestPath will fill the distTo map with shortest path
  //O(1)
  int bestDist = infity
  int candidate = null
  // $O(|V|) = O(|V|) * O(1)$ 
  for each v in graph do
    //O(1)
    if(max(distTo1[v],distTo2[v]) < bestDist)
      then bestDist = max(distTo1[v],distTo2[v])
        candidate = v
    end if
  end for
  return candidate

```

Thus the complexity of findRendezvousPoint is $O(|V|\log|V| + |E|\log|V|)$

Then we prove this algorithm is correct and suppose dijkstraShortestPath works correctly

- i. Since dijkstraShortestPath works correctly and it will fill the distTo map with shortest path, then after execute these four steps

```

Map distTo1 = new_map()
Map distTo2 = new_map()
call dijkstraShortestPath(Graph G, n1)
call dijkstraShortestPath(Graph G, n2)

```

We are guaranteed that distTo1 is the map s.t. shortest path from n1 to that node is stored in it correctly. The same for distTo2

- ii. Then we execute those steps

```

int bestDist = infty
int candidate = null
for each v in graph do
    if(max(distTo1[v],distTo2[v]) < bestDist)
        then bestDist = max(distTo1[v],distTo2[v])
            candidate = v
    end if
end for

```

In the beginning, we assume `bestDist` is infinity and `candidate` is null

We want to find the node R such that the time it takes for both teams is minimized, this is equivalent to finding the node $R =$

v , where v satisfy $\max(\text{distTo1}[v], \text{distTo2}[v]) \leq \max(\text{distTo1}[u], \text{distTo2}[u]), \forall u \in \text{graph}$.

Because two teams go to R parallelly, then the time of meeting at R is the maximum of two teams' cost time. And suppose the speed of two teams are same, then we only need to find the minimum of maximum of two teams' distance to R

To find minimum of maximum of two teams' distance to R , we need two teams' distance to R are both shortest. This is guaranteed by the algorithm of `dijkstraShortestPath`.

Thus we only need to compare and find the minimum one.

Then we use loop invariant theorem to prove this loop will preserve the invariant.

- P_c : `distTo1` and `distTo2` are maps containing the shortest path from $n1$ and $n2$ to $v \in V, \forall v$. `bestDist` is ∞ . `candidate` is `null`.
- Inv : Let S be the set of vertices that are visited before. `bestDist` = $\min_{u \in S}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ and `candidate` is a vertex in S that correspond to the minimum of `bestDist`
- B (loop condition): There are still vertices that are not visited
- Q_c : `candidate` is R such that $\max(\text{distTo1}[R], \text{distTo2}[R]) = \min_{v \in V}(\max(\text{distTo1}[v], \text{distTo2}[v]))$.

a. Initialization ($P_c \Rightarrow Inv$)

At first, the $\max(\text{distTo1}[v], \text{distTo2}[v])$ must be less than infinity since the graph is connected and not empty. Then `bestDist` is $(\max(\text{distTo1}[u], \text{distTo2}[u]))$ which is also $\min_{u \in S}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ since current $S = \{v\}$. And `candidate` are updated to v which corresponds to that `bestDist`

b. Maintenance (Assuming $Inv \wedge B$, show Inv holds after loop once)

Suppose $S \subsetneq V$ where V is the set of all vertices in graph.

Suppose `bestDist` = $\min_{u \in S}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ and `candidate` is a vertex in S that correspond to the minimum of `bestDist`

Then since $S \subsetneq V$, we enter the loop. Then $S' = S \cup \{v\}$ where v is the vertex we just visited. The first invariant holds

Then we excute

```

if(max(distTo1[v],distTo2[v]) < bestDist)
    then bestDist = max(distTo1[v],distTo2[v])
    candidate = v

```

If the maximum distance of $n1/n2$ to v is shorter than `bestDist`, we update `bestDist` and `candidate`. Then $\text{bestDist} = \min_{u \in S'}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ and `candidate` is a vertex in S' that correspond to the minimum of `bestDist`

If not, we know that $\text{bestDist} = \min_{u \in S'}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ and `candidate` is a vertex in S' that correspond to the minimum of `bestDist` since the distance to v is longer than `bestDist`

c. Termination ($Inv \wedge \neg B \Rightarrow Qc$)

Suppose $S = V$ where V is the set of all vertices in graph and $\text{bestDist} = \min_{u \in S}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ and `candidate` is a vertex in S that correspond to the minimum of `bestDist`

Then $\text{bestDist} = \min_{u \in V}(\max(\text{distTo1}[u], \text{distTo2}[u]))$ and `candidate` is a vertex in V that correspond to the minimum of `bestDist`

Thus `candidate` is R such that $\max(\text{distTo1}[R], \text{distTo2}[R]) = \min_{v \in V}(\max(\text{distTo1}[v], \text{distTo2}[v]))$.

Thus this algorithm is correct

3. (10 pts) Implement the algorithm in Java. Provide meaningful tests.

Hint: Think how Dijkstra could help.